

Week - 2

Promise in JavaScript

↳ Promise are a javascript feature that provide a more structure and readable way to work with asynchronous code. They represent the eventual completion or failure of a asynchronous operation

Key Characteristic of Promise

1) Asynchronous Operations:

- promises are commonly used to handle async operation, such as fetching data from a server, reading a file, or executing a timer

2) States

- A promise can be in one of three states
 - pending → The initial state, before the promise is resolved or rejected.
 - Fulfilled (Resolved) → The operation completed successfully, and the promise has a resulting value.
 - Rejected → There was an error during the operation and the promise has a reason for the failure

3) Chaining:

- supports chaining through the then method

4) Error handling

- Promises have built-in error handling through the catch method.

Why Do we need Promises?

- 1) Avoiding Callback hell $\rightarrow$ where nesting callbacks leads to unreadable and hard to maintain code.

```
// Without Promises
asyncOperation1((result1) => {
  asyncOperation2(result1, (result2) => {
    asyncOperation3(result2, (result3) => {
      // ...
    });
  });
});

// With Promises
asyncOperation1()
  .then((result1) => asyncOperation2(result1))
  .then((result2) => asyncOperation3(result2))
  .then((result3) => {
    // ...
  });
```

2. Sequential Execution of Asynchronous Code

- Promises provide a clean way to execute asynchronous operations sequentially, improving code readability

```
// Without Promises
asyncOperation1((result1) => {
  asyncOperation2(result1, (result2) => {
    asyncOperation3(result2, (result3) => {
      // ...
    });
  });
});

// With Promises
asyncOperation1()
  .then((result1) => asyncOperation2(result1))
  .then((result2) => asyncOperation3(result2))
  .then((result3) => {
    // ...
  });
```

3. Error handling:

- Promise simplify error handling by providing a centralized catch block to handle errors.

```
asyncOperation1()
  .then((result1) => asyncOperation2(result1))
  .then((result2) => asyncOperation3(result2))
  .catch((error) => {
    console.error('An error occurred:', error);
  });
```

4) Promise.all for Parallel Execution:

- Promises offer the Promise.all method, allowing parallel execution of multiple asynchronous operations and waiting

for all of them to complete.

```
JavaScript Copy

const promise1 = asyncOperation1();
const promise2 = asyncOperation2();

Promise.all([promise1, promise2])
  .then((results) => {
    const result1 = results[0];
    const result2 = results[1];
    // ...
  })
  .catch((error) => {
    console.error('An error occurred:', error);
  });
```

Promise Basics

1. Creating a Promise:

- A promise represents the eventual completion or failure of an asynchronous operation.
- The Promise constructor takes a function with two parameters resolve and reject.

```
const myPromise = new Promise((resolve, reject) => {
  // Asynchronous operation goes here
  // if successful, call resolve with the result
  // if there's an error, call reject with the error
});
```

2. Resolving a Promise:

- use the resolve function when the asynchronous operation is successful.

```
const successfulPromise = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve('Operation succeeded');
  }, 1000);
});
```



```
    }, 1000);  
  });
```

- Similarly we can use the reject function when there's an error during the asynchronous operation.

Consuming Promises:

1) Using then and catch:

- The then method is used to handle the resolved value.
- The catch method is used to handle error.

Successful Promise

```
• then((result) => {  
    console.log(result); // Output: Operation  
                           succeeded!  
})
```

```
• catch((error) => {  
    console.log(error); // This won't be called in  
                           this example.  
});
```

2. Chaining Promises

```
successfulPromise  
  .then((result) => {  
    console.log(result); // Output: Operation succeeded!  
    return 'New value';  
  })  
  .then((newValue) => {  
    console.log(newValue); // Output: New value  
  })  
  .catch((error) => {  
    console.error(error);  
  });
```


3. Promise All

- `Promise.all` is used to wait for multiple promises to complete.

```
const promise1 = Promise.resolve('One');  
const promise2 = Promise.resolve('Two');  
  
Promise.all([promise1, promise2])  
  .then((values) => {  
    console.log(values); // Output: ['One', 'Two']  
  })  
  .catch((error) => {  
    console.error(error);  
  });
```